Problem B

Com a data para o novo formato híbrido aproximando-se, a SAP decidiu construir um novo edifício para comportar os funcionários que passarão a vir mais vezes ao escritório. No projeto de construção deste edifício, existem ao todo N salas de trabalho, conectadas por corredores, de tal forma que é possível trafegar de uma sala para qualquer outra seguindo um caminho entre salas e corredores. Para cada corredor que conecta duas salas, existe um custo Ci para a construção do corredor.

Sua tarefa é determinar qual o menor custo necessário para construir o edifício, dada a configuração do projeto inicial, e a restrição de que na construção final a regra de poder ir de uma sala para outra qualquer passando por uma sequência de salas e corredores deve ser mantida. O custo da construção de uma sala é negligível, apenas o custo da construção dos corredores importa.

Inputs

A primeira linha da entrada contém dois inteiros, N e M ($2 \leq N \leq 103$ e $1 \leq M \leq 105$), sendo o número de salas existentes no projeto do edifício, e o número de corredores, respectivamente. As salas são enumeradas de 1 a N.

Cada uma das M linhas a seguir contém três inteiros Ai, Bi e Ci ($1 \leq Ci \leq 103$), indicando que no projeto inicial do edifício existe um corredor que conecta as salas Ai e Bi e que seu custo de construção é Ci.

Outputs

Seu algoritmo deve imprimir uma única linha com um número inteiro, indicando o menor custo possível para a construção do edifício, dadas as restrições impostas.

Examples

Exemplo de entrada 1	Exemplo de saída 1
4 4	60
1 2 10	
1 3 40	
2 4 30	
3 4 20	

Exemplo de entrada 2	Exemplo de saída 2
5 6	
1 2 8	
2 3 12	
2 4 9	
2 5 3	

Exemplo de entrada 2	Exemplo de saída 2
3 4 5	
4 5 1	

Code

```python
# Estrutura de dados utilizada: lista de arestas e estrutura de união-busca (Union-Find)
# Tipo de algoritmo: Algoritmo de Kruskal para Árvore Geradora Mínima (MST)

def find(parent, x):
    if parent[x] != x:
        parent[x] = find(parent, parent[x])
    return parent[x]

def union(parent, rank, x, y):
    xroot = find(parent, x)
    yroot = find(parent, y)
    if xroot == yroot:
        return False
    if rank[xroot] < rank[yroot]:
        parent[xroot] = yroot
    else:
        parent[yroot] = xroot
        if rank[xroot] == rank[yroot]:
            rank[xroot] += 1
    return True

N, M = map(int, input().split())
edges = []
for _ in range(M):
    a, b, c = map(int, input().split())
    edges.append((c, a-1, b-1))

edges.sort()
parent = [i for i in range(N)]
rank = [0] * N
cost = 0
for c, a, b in edges:
    if union(parent, rank, a, b):
        cost += c

print(cost)
```